



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA



UCSC



UNIVERSIDAD DEL BÍO-BÍO

Knowlegde distillation and NVIDIA TensorRT

Part I

Doctorado en Inteligencia Artificial

Profesor: Jorge Portilla G.

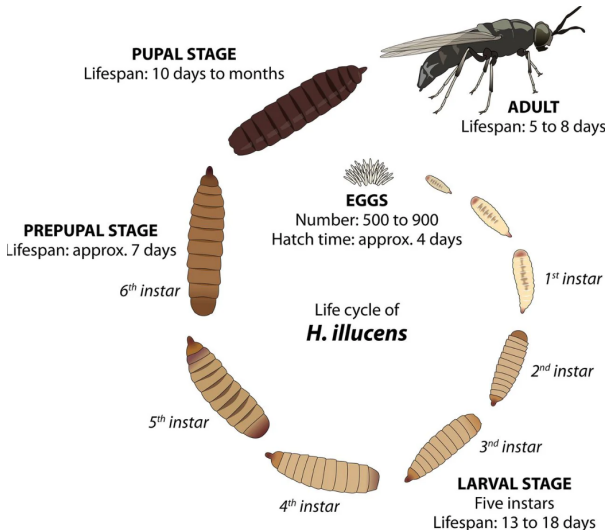
Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

1 Performance Assessment

- Latency
- Throughput
- Size of model

2 Soot Pyrometry

3 Knowledge distillation

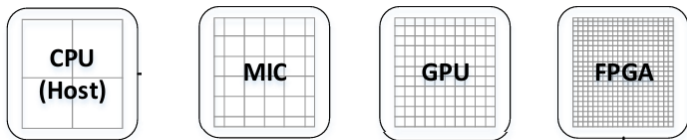


*"Many insects have a larval form that is optimized for extracting energy and nutrients from the environment and a completely different adult form that is optimized for the very different requirements of traveling and reproduction."*¹

¹G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network", arXiv preprint arXiv:1503.02531, 2015.

El hardware se refiere al tipo específico de dispositivo en el que se ejecutará el modelo de *machine learning* durante la fase de **inferencia**.

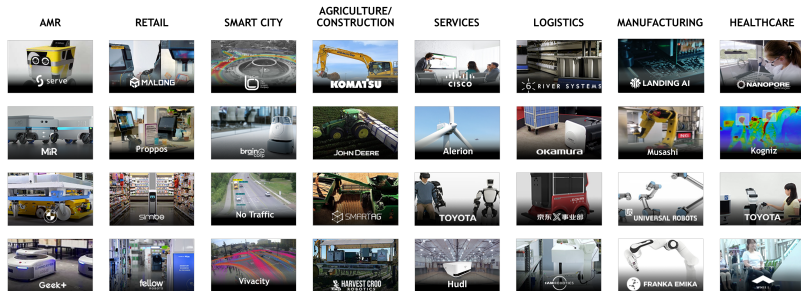
- ▶ Este puede variar ampliamente según las necesidades de la aplicación, el entorno de despliegue y los requisitos de rendimiento y eficiencia.
- ▶ Los tipos de hardware especializado incluyen:
 - **GPUs** (Graphics Processing Units)
 - **CPUs** (Central Processing Units)
 - **FPGAs** (Field-Programmable Gate Arrays)
 - Aceleradores específicos como:
 - **TPUs** (Tensor Processing Units)
 - **NPU**s (Neural Processing Units)
- ▶ Entre estas opciones, las **GPUs** destacan como las más accesibles y populares debido a sus capacidades de cómputo paralelo, esenciales para las aplicaciones de ML.



- ▶ La *Central Processing Unit* (CPU) convencional, junto con cualquier combinación de aceleradores especializados que ofrecen granularidad mixta para procesamiento paralelo².
- ▶ Los aceleradores especializados incluyen:
 - **Many Integrated Cores (MIC)** processors³.
 - **Graphics Processing Units (GPUs)**.
 - **Field-Programmable Gate Arrays (FPGAs)**.
- ▶ Estas tecnologías representan la mayor parte de la innovación en hardware disponible hoy en día para aplicaciones de ML.

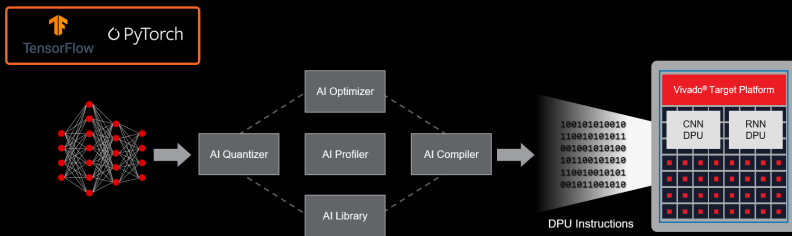
²La unidad de cómputo es capaz de dividir las tareas en porciones más pequeñas y procesarlas en paralelo.

³Intel Xeon Phi, son procesadores diseñados para el cómputo paralelo masivo. Su arquitectura se basa en múltiples cores para procesar tareas de manera simultánea.



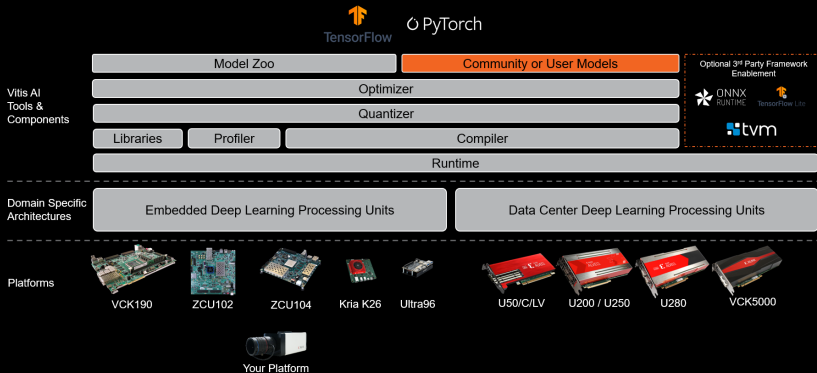
- ▶ *Safe-critical*
- ▶ *Resource-constrained devices*

Deployment in the Vitis™ AI Environment – A 1000 Foot View

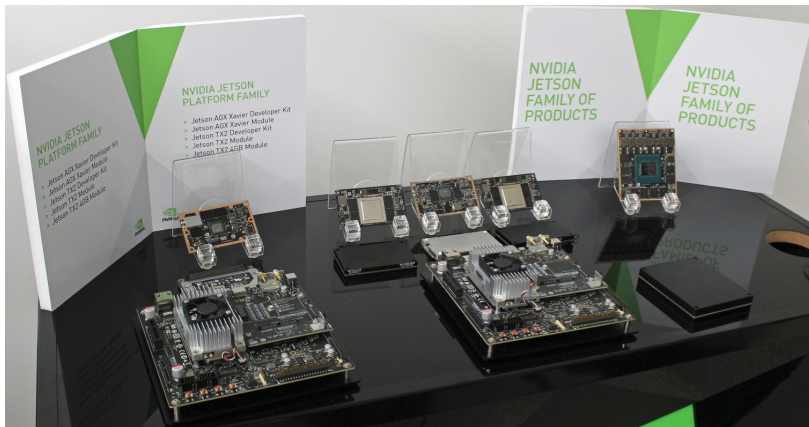


- **Vitis AI** es un conjunto de herramientas de Xilinx (ahora AMD) diseñado para implementar modelos de inteligencia artificial en FPGAs y SoCs, especialmente en la serie Xilinx Zynq Ultrascale+ MPSoC.

Vitis™ AI Integrated Development Environment



Jetson - Características de los dispositivos



Plataforma	Núcleos GPU	Memoria (GB)	CPU
Nvidia GeForce RTX 3060	—	12 (GDDR6) + 36 RAM	Intel i3-12100F
Nvidia GeForce RTX 2060	—	6 (GDDR6) + 12 RAM	Intel i7-10750H
Jetson Xavier NX	384	8 RAM (compartida)	ARMv8.2, 6 núcleos, 1.9 GHz
Jetson Xavier AGX	512	16 RAM (compartida)	ARMv8.2, 8 núcleos, 2.3 GHz
Jetson Orin Nano	1024	8 RAM (compartida)	ARMv8.2, 6 núcleos, 1.7 GHz
Jetson Orin AGX	2048	64 RAM (compartida)	ARMv8.2, 12 núcleos, 2.2 GHz

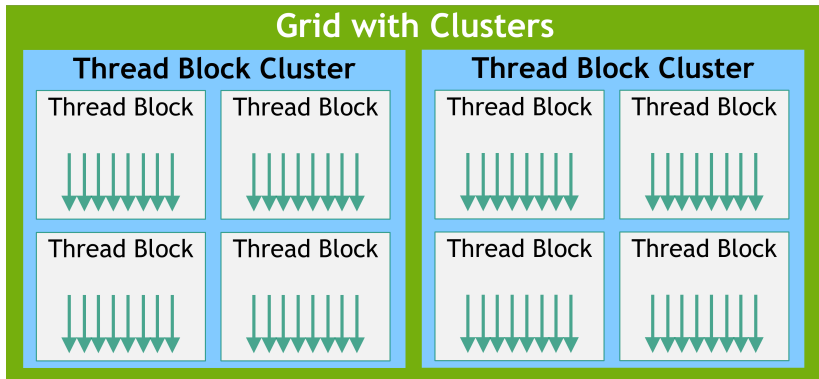
- ▶ CUDA es una plataforma computacional y modelo de programación desarrollado por Nvidia.
- ▶ Permite ejecutar tareas altamente paralelizables en GPUs de Nvidia.
- ▶ Se basa en el concepto de *General-Purpose Computing on GPUs* (GPGPU).
- ▶ Compatible con lenguajes como C/C++, Fortran y Python.

- ▶ **Acelera aplicaciones de deep learning y cómputo científico.**
- ▶ **Permite optimización mediante paralelismo masivo.**

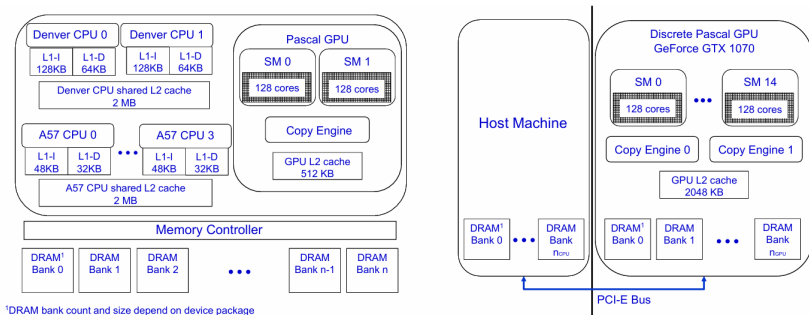
Conceptos Claves en CUDA

- ▶ **Hilos (Threads):** Unidades de ejecución paralela.
- ▶ **Bloques de Hilos (Thread Blocks):** Agrupaciones de hilos que ejecutan tareas conjuntas.
- ▶ **Grid:** Conjunto de bloques de hilos.
- ▶ **Memoria Global y Compartida:** Estrategias de almacenamiento para optimizar el rendimiento.
- ▶ **GPUs Discretas:** Son chips separados de la CPU, conectados mediante buses como PCIe (común en PCs de escritorio).
- ▶ **GPUs Integradas:** Están en el mismo chip que la CPU, compartiendo memoria y recursos.

Grid with Clusters



- ▶ **Hilos (Threads):** Unidades de ejecución paralela.
- ▶ **Bloques de Hilos (Thread Blocks):** Agrupaciones de hilos que ejecutan tareas conjuntas.
- ▶ **Grid:** Conjunto de bloques de hilos.



- Diagrama de comunicación entre *host* y *device*: GPU integrada (izquierda) y GPU discreta (derecha)⁴.
- CUDA es un modelo de programación heterogéneo que utiliza tanto la CPU como la GPU.

⁴Ming Yang, et al, "Avoiding Pitfalls when Using NVIDIA GPUs for Real-Time Tasks in Autonomous Systems," *Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium*, 2021.

Siguiendo la nomenclatura ampliamente utilizada y aceptada en el contexto de GPUs, nos referiremos al procesador principal como *host* y al coprocesador como *device*.

- ▶ Existen distintos elementos clave en el procesamiento paralelo dentro de una GPU:
 - CPU Cores
 - GPU Cores
 - Streaming Multiprocessor

► CPU Cores

- Unidades de procesamiento de una CPU.
- Especializados en ejecutar tareas secuenciales con alta eficiencia.

► GPU Cores

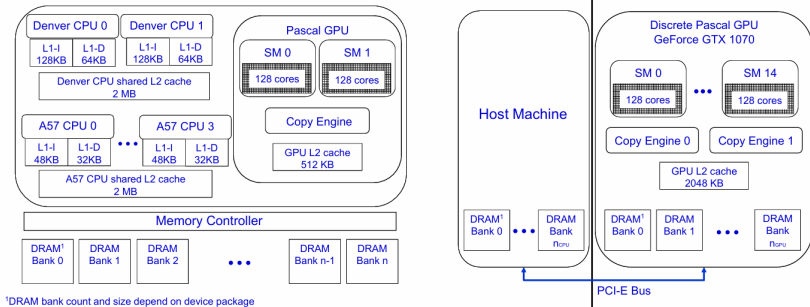
- Unidades de procesamiento dentro de una GPU.
- Optimizados para operaciones en paralelo, ideales para procesamiento masivo de datos⁵.
- No pueden ejecutar sistemas operativos ni gestionar tareas autónomamente.
- Funcionan como coprocesadores bajo instrucciones de la CPU.

► Streaming Multiprocessor (SM)

- Unidad dentro de las GPUs de Nvidia que agrupa múltiples GPU Cores.
- Ejecuta múltiples hilos de procesamiento simultáneamente.
- Coordina la ejecución paralela de un gran número de operaciones.

⁵como el renderizado de gráficos o algoritmos de machine learning

Elementos del Procesamiento Paralelo en GPUs



► Implementación *system-on-chip* (SoC).

- Esta integra la GPU, los *CPU Cores*, la memoria (*Unified Memory*) y los conectores externos en una pequeña computadora de una sola placa. La GPU integrada comparte recursos de hardware, como la memoria, con los *CPU Cores*.

► GPU discreta

- GPUs discretas, consisten únicamente en los *Streaming Multiprocessors* (SMs) y la memoria local del dispositivo (*device memory*).
- Empaquetados en una tarjeta para su instalación en una ranura de expansión PCIe de la placa base de una computadora.

Niveles populares de software y hardware.

Capa	Ejemplos
Aplicación	Visión por computadora Control Automático Reconocimiento de voz Conducción autónoma
Modelo	ResNet - MobileNet - MobileViT YOLO - U-Net
Framework	PyTorch - TensorFlow - Keras
Graph Format	ONNX NNEF ⁶
Inference Optimizer	TensorRT ARM Compute Library Intel OpenDNN AMD ML Open Xilinx Vitis AI
Hardware	GPU CPU FPGA Aceleradores especializados (TPU, NPU, etc.)

⁶Sintaxis similar a JSON o Python, lo que facilita su comprensión y edición manual

Funcionamiento de CUDA y Kernels

- ▶ Un programa CUDA inicia su ejecución en el **host** (CPU). El **device** (GPU) realiza las tareas de cálculo.
- ▶ **Pasos generales de un programa CUDA:**
 1. Asignación de memoria en la GPU.
 2. Transferencia de datos de la CPU a la GPU.
 3. Ejecución de **kernels** en la GPU.
 4. Copia de resultados de la GPU a la CPU.
 5. Liberación de memoria.
- ▶ **Conceptos importantes:**
 - **Kernels:** Funciones que se ejecutan en la GPU.
 - **Kernel launch:** Proceso de ejecución de un kernel.
- ▶ **Rendimiento:**
 - Cada kernel launch introduce retardos.
 - Optimización:
 - Reducir kernel launches.
 - Seleccionar kernels adecuados.
 - **Inference optimizers** realizan estas optimizaciones.

- ▶ Estudios sobre GPUs Nvidia y CUDA revelan que aspectos críticos como:
 - Planificación de tareas.
 - Sincronización.
 - Ejecución.
 - Bloqueo de tareas.
- ▶ No están completamente documentados ni son públicamente accesibles.
- ▶ Naturaleza de “caja negra” de la GPU:
 - La falta de documentación convierte el comportamiento interno de la GPU en una “caja negra” para los desarrolladores.
 - Los desarrolladores deben depender de **evidencia empírica** para inferir cómo se comportan los programas CUDA.



- ▶ Documentación oficial de CUDA de Nvidia.
- ▶ Libro: *Programming Massively Parallel Processors* de Kirk y Hwu.
- ▶ Tutoriales en línea y cursos especializados en computación paralela.

Performance Assessment

- ▶ Existen dos métricas principales para caracterizar el rendimiento de un sistema digital en términos de "*speed*".
 - **Latency (response time or execution time)**⁷⁸
 - Cuánto tiempo se tarda en generar un resultado a partir de un conjunto de entrada.
 - Cuanto tarda en completar una tarea.
 - **Throughput (bandwidth)**
 - Número de resultados producidos por unidad de tiempo.
 - Típicamente relevante para datacenters-

⁷ Medida importante para servicios basados en IA (Alexa y Siri)

⁸ Aplicaciones *safe-critical*

- En general, nuestro objetivo es mejorar el rendimiento de una computadora A reduciendo el tiempo de ejecución.

$$Performance_A = \frac{1}{Execution\ time_A}$$

- Cuando decimos: "el sistema A es n veces más rápido que el sistema B "

$$\frac{Performance_A}{Performance_B} = \frac{Execution\ time_B}{Execution\ time_A} = n$$

- ▶ La latencia se define como el tiempo necesario para procesar una única entrada a través de la red con un *batch size* de inferencia igual a uno.
- ▶ Generalmente, se excluye el preprocesamiento de la entrada.
- ▶ En los experimentos, se mide usando rondas de *warm-up*⁹ equivalentes al 10 % del total de datos del conjunto de validación.
- ▶ El *warm-up* permite estabilizar el sistema antes de la medición, evitando valores atípicos debido a la carga inicial del modelo.

⁹pruebas iniciales o enviar tráfico de prueba antes de comenzar las mediciones de latencia reales

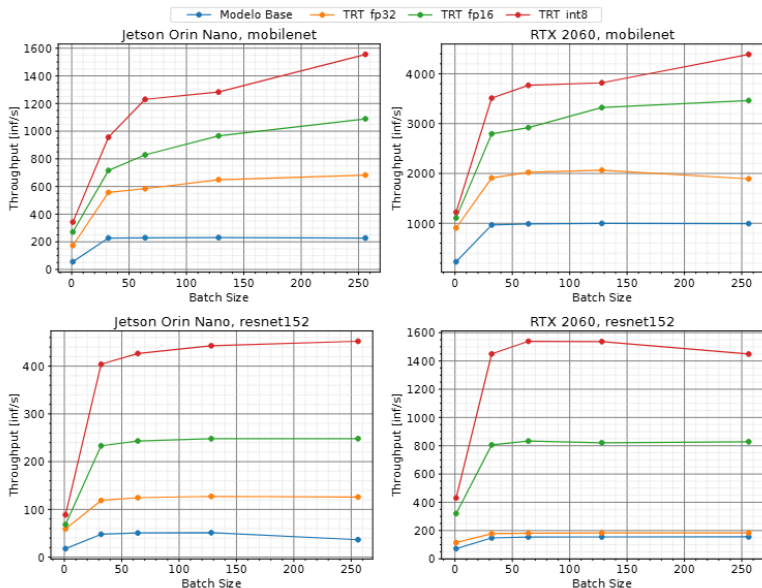
- ▶ La latencia se mide calculando el tiempo total de ejecución de cada inferencia.
- ▶ Se incluyen transferencias de datos entre la CPU y la GPU, y viceversa.
 - *Overhead* asociado con la gestión de la memoria.
- ▶ En python, se puede utilizar la función `time` del módulo `time` de Python.
- ▶ Se registran valores de latencia para una única ejecución en todo el conjunto de validación.
- ▶ Se calcula la latencia promedio y la latencia máxima.

- ▶ El *throughput* indica la cantidad de datos procesados por unidad de tiempo.
- ▶ Se puede calcular dividiendo el número de entradas del modelo (*batch size* de inferencia) por el tiempo necesario para procesarlas.
- ▶ El tiempo de procesamiento se mide de la misma manera que la latencia, usando un *warm-up*¹⁰ del 10 %.
- ▶ Se incluyen los tiempos de transferencia de datos entre la CPU y la GPU.

¹⁰Unas inferencias iniciales antes de las mediciones

- ▶ Se registran los tiempos de procesamiento de cada lote de entradas con un tamaño de *batch size* específico.
- ▶ Se calcula un tiempo de procesamiento promedio para determinar el *throughput* promedio.
- ▶ Una recomendación para evaluar *throughput*, es evaluar distintos tamaños de *batch size*, comenzando con 1, luego 32 y duplicándolo hasta 256.
- ▶ Aumentar el *batch size* puede incrementar el *throughput*.

Medición de Throughput

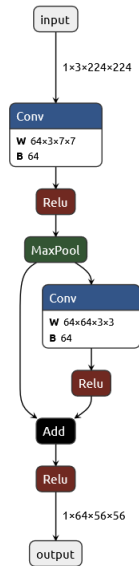


¹¹Comparación del throughput en función del batch size de inferencia para los modelos MobileNet y ResNet152 en las plataformas Jetson Orin Nano y RTX 2060.

- ▶ El tamaño del modelo y la capacidad de la red entre procesadores influyen en el rendimiento.
- ▶ Hay varios factores en el aprendizaje profundo, como el número de capas, el número de neuronas por capa, la complejidad del cálculo por capa y el número de conexiones entre un nodo de una capa y los nodos de las capas siguientes.

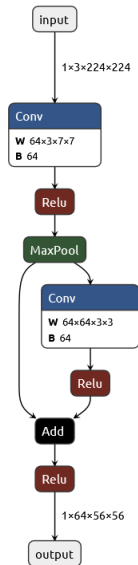
Graph Format en Machine Learning

- Un modelo de Machine Learning se representa como un **grafo computacional**, facilitando su interoperabilidad entre frameworks.
- Permite visualizar cómo los datos se procesan a través de las capas del modelo.



Elementos del Grafo Computacional

- **Nodos:** Representan capas u operaciones del modelo. Ejemplo: convolucional (azul).
- **Aristas:** Simbolizan los tensores y muestran el flujo de datos entre nodos.
- **Parámetros:** Son los pesos (W) y sesgos (B), ajustados en el entrenamiento.



- ▶ El tamaño del modelo y el ancho de banda de la red entre procesadores afectan el rendimiento¹².
- ▶ Factores clave en el aprendizaje profundo:
 - **Número de capas:** Más capas aumentan la complejidad del modelo.
 - **Neuronas por capa:** Determina la capacidad de representación.
 - **Complejidad computacional por capa:** Afecta el tiempo de entrenamiento e inferencia.
 - **Conexiones entre capas:** Más conexiones implican un mayor costo computacional.

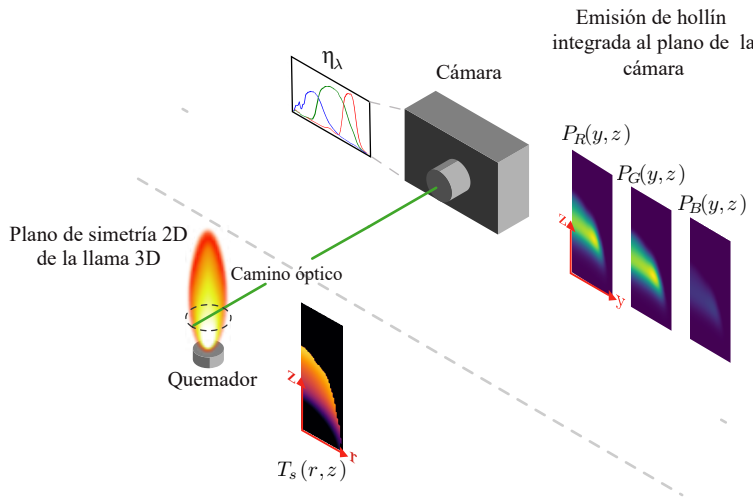
Conclusión: Optimizar el tamaño y la arquitectura del modelo es crucial para equilibrar precisión y eficiencia.

¹²En sistemas con PCIe, NVLink, o Ethernet, un ancho de banda bajo puede causar cuellos de botella.

Tamaño del Modelo (Model Size)

- ▶ Se refiere al tamaño total ocupado por los parámetros y la estructura del modelo.
- ▶ Se mide en términos de la cantidad de memoria necesaria para almacenarlo (en megabytes).
- ▶ La medición se puede realizar utilizando la función `getsize` del módulo `os` de Python.
 - Métodos como cuantización, *pruning* y *knowledge distillation* pueden reducir significativamente el tamaño del modelo.
- ▶ Modelos almacenados en `fp32` ocuparán más espacio que en `fp16` o `int8` (cuantización).

Soot Pyrometry



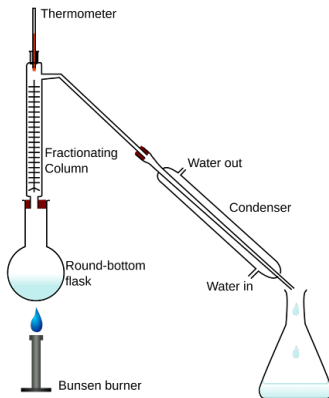
Esquema del arreglo óptico BEMI para capturar la radiación emitida por las partículas de hollín.

Tabla 1: Resultados de latencia para un batch size de inferencia de 1 y de throughput para batch sizes de inferencia entre 2 y 32 en la plataforma Jetson Orin AGX.

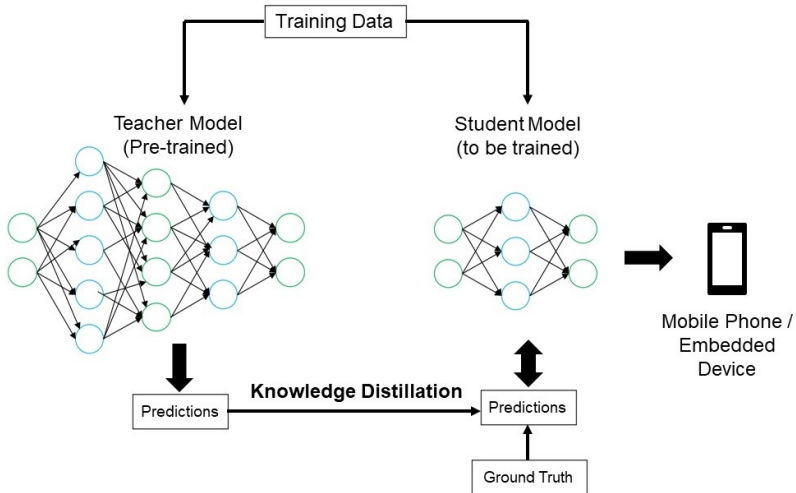
	Modelo	Latencia [ms]		Throughput [inf/s]				
		ave.	max.	2	4	8	16	32
U-Net	Modelo Base	126.5	714.5	27.17	31.29	81.33	117.49	208.05
	TRT fp16	112.2	139.7	35.18	52.53	125.72	135.41	268.36
Att. U-Net	Modelo Base	134.2	753.8	15.20	30.97	58.53	92.60	122.94
	TRT fp16	115.4	312.8	19.74	42.10	65.11	123.34	220.30

Knowledge distillation

What is knowledge distillation?



Knowledge distillation is a machine learning technique that aims to transfer the learnings of a large pre-trained model, the “teacher model,” to a smaller “student model” (*light-weight neural network*). It’s used in deep learning as a form of model compression and knowledge transfer, particularly for massive deep neural networks.



Training and inference have difference requirements

Técnica para transferir conocimientos de un modelo grande (teacher) a un modelo más pequeño (student).

- ▶ Procesamiento del Lenguaje Natural (NLP)
- ▶ Reconocimiento de voz
- ▶ Reconocimiento y detección de imágenes
- ▶ Modelos de Lenguaje Grandes (LLMs)
- ▶ Modelos más **eficientes y accesibles**.
- ▶ Reducción del **costo computacional** sin perder precisión.

Permite a modelos pequeños **aproximar** el rendimiento de modelos grandes. **Aplicaciones en diversas áreas.**

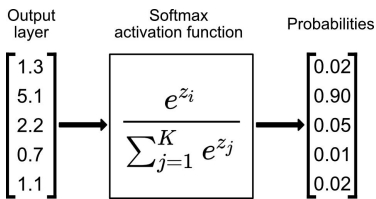
Objetivo: Modelos de IA más ligeros y rápidos sin comprometer su precisión.

Knowledge Distillation y Softmax con Temperatura

- ▶ **Teacher Model:** Produce una distribución suave.
- ▶ **Student Model:** Se entrena con estas distribuciones suaves (soft targets) en lugar de etiquetas duras.
- ▶ **Ventaja:** Captura patrones más ricos y generalizables sin perder información de clases cercanas.

Knowledge Distillation y Softmax con Temperatura

- **Teacher Model:** Produce una distribución suave.
- **Student Model:** Se entrena con estas distribuciones suaves (soft targets) en lugar de etiquetas duras.
- **Ventaja:** Captura patrones más ricos y generalizables sin perder información de clases cercanas.



- La temperatura elevada permite que el *teacher model* produzca *soft targets* más suaves.
- El *student model* se ajusta a esos *soft targets*, aprendiendo sobre relaciones entre clases.

Función Softmax:

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

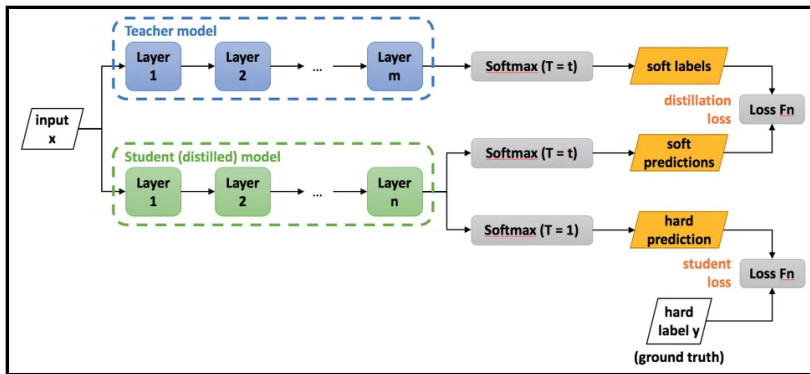
- ▶ **Logits** z_i : Salidas del modelo para cada clase.
- ▶ Softmax resalta los valores grandes pero pierde "*relativeness*" con las otras clases.

Softmax con Temperatura:

$$P(y_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

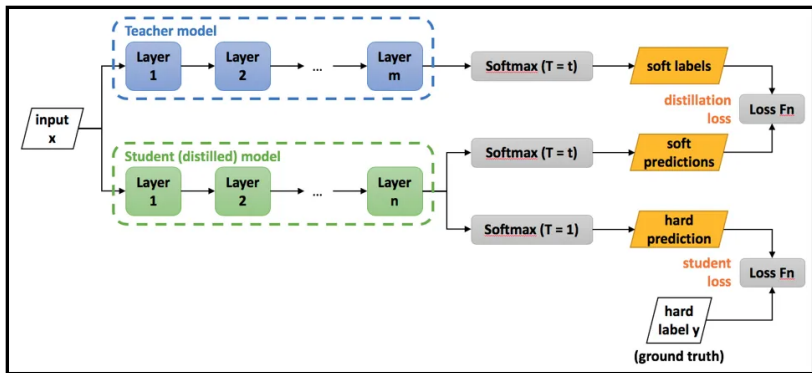
- ▶ $T > 1$: Suaviza la distribución de probabilidades.
- ▶ El modelo es menos confiado, considerando más clases con probabilidades similares.

Terminología en KD

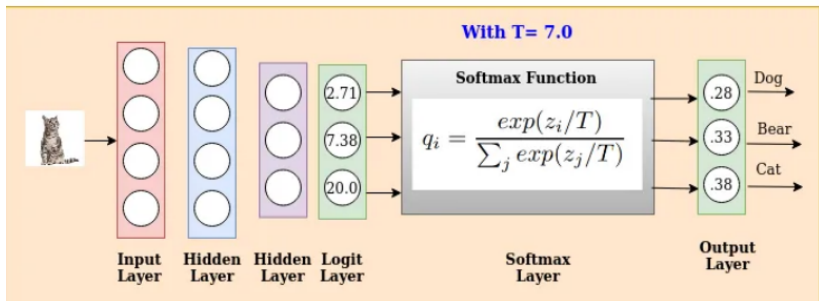
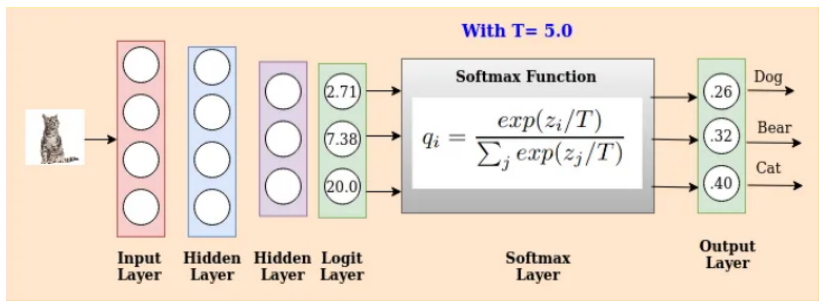


- ▶ **Teacher model**: El modelo original (cumbersome) se llama modelo maestro, ya que estamos extrayendo conocimiento de él.
- ▶ **Student model**: El nuevo modelo con menos parámetros se llama modelo estudiante, ya que estamos destilando información en él.
- ▶ **Soft labels**: La salida del modelo maestro donde se usa softmax con una temperatura mayor que 1 ($T > 1$).

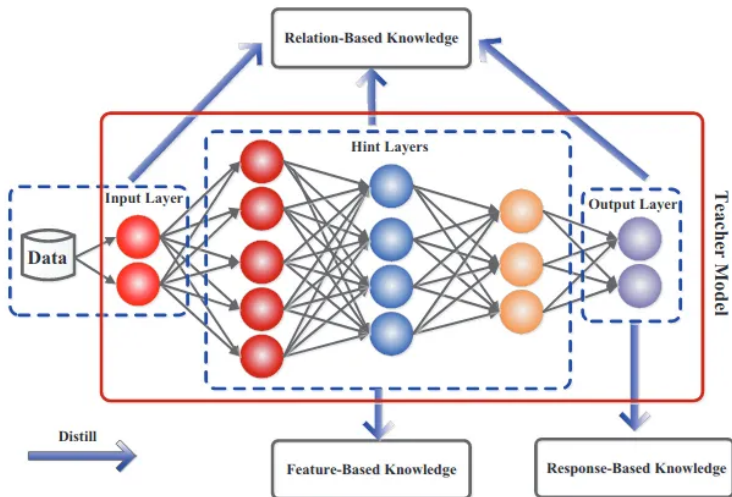
Terminología en KD



- ▶ **Soft labels:** La salida del modelo maestro donde se usa softmax con una temperatura mayor que 1 ($T > 1$).
- ▶ **Soft predictions:** La salida del modelo estudiante donde se usa softmax con una temperatura mayor que 1 ($T > 1$).
- ▶ **Hard predictions:** Cuando se usa el softmax regular en el modelo estudiante.
- ▶ **Hard labels:** La etiqueta de verdad de referencia en forma de **vector one-hot**.



Types of Knowledge



Existen diferentes enfoques para transferir conocimiento de un modelo grande (*teacher*) a uno más pequeño (*student*):

► Response-based KD (Logit-based)

- Se basa en la comparación de los **logits** (salidas suavizadas del softmax) entre el teacher y el student.
- Se usa la **divergencia de Kullback-Leibler (KL)** para minimizar la diferencia entre las distribuciones de salida.

► Feature-based KD

- Se enfoca en transferir representaciones intermedias en lugar de solo los logits de salida.
- Se minimiza la diferencia entre los mapas de características aprendidos por el teacher y el student.

► Relation-based KD¹³

- En lugar de transferir logits o características, se transfieren las **relaciones entre ejemplos**.
- Se entrena el student para replicar la estructura geométrica del espacio de características del teacher.

¹³ cada entrada x_i se transforma en una representación en un espacio de características, y se emplean relación geométricas como distancias y similitudes entre cosenos.

- ▶ Primero, entrenar el *student* y *teacher networks* de forma independientes.
- ▶ El método funciona incorporando una función de pérdida adicional a la pérdida de *cross-entropy* tradicional, que se basa en la salida softmax de la red *teacher*.
- Se puede suponer que la información valiosa no sólo esta presente en la predicción más alta de un modelo correctamente entrenado, sino en toda la distribución de salida.

Para entrenar el *student*, se requiere la combinación de dos términos en la función de costo:

1. **Pérdida por entropía cruzada (CE)**: evalúa la diferencia entre la salida del estudiante y las etiquetas reales.
2. **Pérdida por destilación (Soft Targets)**: mide la diferencia entre las salidas suavizadas del *teacher* y el *student*.

La función de pérdida en KD es:

$$L = \lambda_{ce} \cdot \mathcal{L}CE(y, \hat{y}) + \lambda_{kd} \cdot T^2 \cdot KL(\sigma(z_T/T), \sigma(z_S/T)) \quad (1)$$

donde:

- ▶ $\mathcal{L}CE(y, \hat{y})$ es la entropía cruzada entre las etiquetas reales y las predicciones del modelo *teacher*.
- ▶ $KL(\sigma(z_T/T), \sigma(z_S/T))$ es la divergencia de Kullback-Leibler entre las distribuciones del maestro y el estudiante, con temperatura T .
- ▶ Pesos λ_{ce} y λ_{kd} .

Consideremos un objetivo que se enfoque en los estados ocultos del estudiante, en lugar de solo en las capas de salida.

- ▶ Transferir información desde la representación interna del *teacher* hacia los estados ocultos del *student*.
- ▶ Se puede utilizar una función de pérdida simple para minimizar la diferencia entre los estados ocultos aplanados.
- ▶ Solo se actualizan los pesos del *student*, ya que el modelo del *teacher* no se actualiza.
- ▶ La idea es que el teacher tiene una mejor representación interna que el estudiante no puede replicar fácilmente sin ayuda.
- ▶ Empujar al estudiante a imitar la representación interna del *teacher* puede mejorar la precisión en la prueba, o potencialmente puede dañar el rendimiento debido a diferencias de arquitectura.
- ▶ No es necesario que los estados ocultos del student coincidan exactamente con los del teacher.

Cosine Embedding Loss mide la similitud entre dos vectores usando el coseno del ángulo entre ellos. Se define como:

$$\mathcal{L}(x_1, x_2, y) = \begin{cases} 1 - \cos(x_1, x_2), & \text{si } y = 1 \\ \max(0, \cos(x_1, x_2) - m), & \text{si } y = -1 \end{cases}$$

Donde:

- ▶ x_1, x_2 son los vectores que se comparan.
- ▶ $y = 1$ si los vectores deben ser similares.
- ▶ $y = -1$ si los vectores deben ser diferentes.
- ▶ $\cos(x_1, x_2)$ es la similitud coseno entre los vectores¹⁴.
- ▶ m es un margen que controla la penalización en caso de diferencia.

En KD, se usa para forzar al modelo estudiante a aprender representaciones similares al modelo maestro (*teacher*) en capas ocultas.

¹⁴Calcular la similitud coseno como el producto punto de `output_normalizado` y `target_normalizado`

- ▶ Generalmente, los métodos de KD no aumentan el número de parámetros de la red o el tiempo de inferencia.
- ▶ El aumento en el rendimiento viene con un costo mínimo al calcular gradientes durante el entrenamiento.
- ▶ En aplicaciones de ML, nos importa principalmente el tiempo de inferencia, ya que el entrenamiento ocurre antes del despliegue del modelo.
- ▶ Se pueden aplicar pérdidas adicionales en muchas tareas, no solo en clasificación.
- ▶ Se pueden experimentar con diferentes parámetros como los **coeficientes**, **temperatura** o **número de neuronas**.

Knowledge Distillation Tutorial

- ▶ Fecha de presentación oral: 15 de Abril (~ 12 minutos y 5 minutos de preguntas)
- ▶ Fecha de entrega informe escrito y código: 20 de Abril (23:59 pm)
 - Informe en formato de paper de conferencia IEEE.
 - Resumen - Métodos - Resultados - Conclusiones - Referencias
 - Figuras y/o tablas para mostrar resultados.
 - Escrito en inglés.
 - Mínimo 3 paginas (sin incluir bibliografía).



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Knowledge distillation and NVIDIA TensorRT

Part I

Doctorado en Inteligencia Artificial

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile